

Matrizes

Algumas estruturas não conseguem ser representadas por **listas** convencionais (vetores 1d). Em alguns momentos precisaremos de listas de $2, 3, \dots, n$ dimensões.

```
l = [1, 2, 3, 4, 5, 6]
```

As matrizes podem ser enxergadas como
vetores bi-dimensionais.

Estruturas que se utilizam de matrizes:

- Matrizes (*matemáticas*)
- Tabuleiros de jogos (ex.: xadrez, damas, batalha naval etc.)
- Pixels de um monitor (*imagens*)
- Palavras Cruzadas
- Etc.

“ O uso de matrizes geralmente ocorre quando precisamos de **linhas** e **colunas** para **identificar** algum elemento. ”

Representação Matemática

$$\begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}$$

linha x coluna

a_{11} é o 1º elemento da 1ª linha.

a_{12} é o 2º elemento da 1ª linha.

a_{21} é o 1º elemento da 2ª linha.

a_{23} é o 3º elemento da 2ª linha.

a_{mn} é o nº elemento da mª linha.

Representação em Python

Ex: matriz *quadrada* 3x3

```
m = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]
```

Representação em Python

Ex: matriz *quadrada* 3x3

```
m = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

“ **Não** existe um tipo básico `matrix` em python. Utilizamos uma lista com várias listas dentro para representar uma matriz. Uma matriz em python é uma lista de listas.

”

Atenção

Nem toda *lista de listas* será uma matriz.

A coleção abaixo não pode ser considerada
uma matriz.

```
l = [[1, 2, 3], [4, 6], [7, 8, 9]]
```

Todas as *linhas* precisam ter o mesmo número de *colunas*. A coleção abaixo é apenas uma **lista de listas**.

```
l = [[1, 2, 3], [4, 6], [7, 8, 9]]
```

Iterando por uma matriz

Iterando pelas linhas

```
mtx = [[1, 2, 3], [4, 5, 6]]  
m = len(mtx)  
for i in range(m):  
    print(mtx[i])
```

Iterando pelas linhas e colunas

Devemos utilizar loops aninhados para percorrer as combinações linha x coluna.

```
mtx = [[1, 2, 3], [4, 5, 6]]  
for i in range(m):  
    for j in range(n):  
        print(mtx[i][j])
```

Recebendo uma matriz como entrada.

Para receber uma matriz como entrada
devemos saber o tamanho $m \times n$ desejado.

A partir do tamanho, podemos definir dois loops aninhados:

```
for i in range(m): # loop das linhas
    for j in range(n): # loop das colunas
        ...
```

Preenchemos com os valores das linhas e colunas

```
mtx = [] # lista de fora
for i in range(m): # loop das linhas
    linha = []
    for j in range(n); # loop das colunas
        coluna = input()
        linha.append(coluna)
    mtx.append(linha)
```

Exercício

Exiba uma matriz na tela de maneira formatada.

Ex. :

1	2	3
4	5	6

Exercício

Receba uma matriz $m \times n$ a partir do input do usuário.

Entradas:

- 1 . m
- 2 . n
- 3 . $mtx[i][j]$

Exercício

Determine a soma dos elementos da diagonal de uma matriz $m \times n$.

Entradas:

1. m
2. n
3. $mtx[i][j]$